Zara Ahmed

FA25-BCS-060

Task#01:

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node* next;
};

class Queue
{
   Node* front;
   Node* rear;

public:
   Queue()
   {
      front = rear = NULL;
```

```
}

bool is_empty()
{
   return front == NULL;
}

void enqueue(int value)
{
   Node* p = new Node;
   p->data = value;
   p->next = NULL;

   if (is_empty())
      front = rear = p;
   else
   {
      rear->next = p;
      rear = p;
   }

   cout << value << " inserted.\n";
}
```

```
int dequeue()
{
   if (is_empty())
   {
      cout << "Queue is empty!\n";
      return -1;
   }

   Node* p = front;
   int value = p->data;

   front = front->next;

   if (front == NULL)
      rear = NULL;

   delete p;
   return value;
}

void display()
{
```

```
    if (is_empty())
    {
        cout << "Queue is empty!\n";
        return;
    }

    Node* p = front;

    while (p != NULL)
    {
        cout << p->data << " ";
        p = p->next;
    }

    cout << endl;
}

~Queue()
{
    while (front != NULL)
    {
        Node* p = front;
        front = front->next;
```

```
            delete p;
        }

        rear = NULL;
    }
};

int main()
{
    Queue q;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

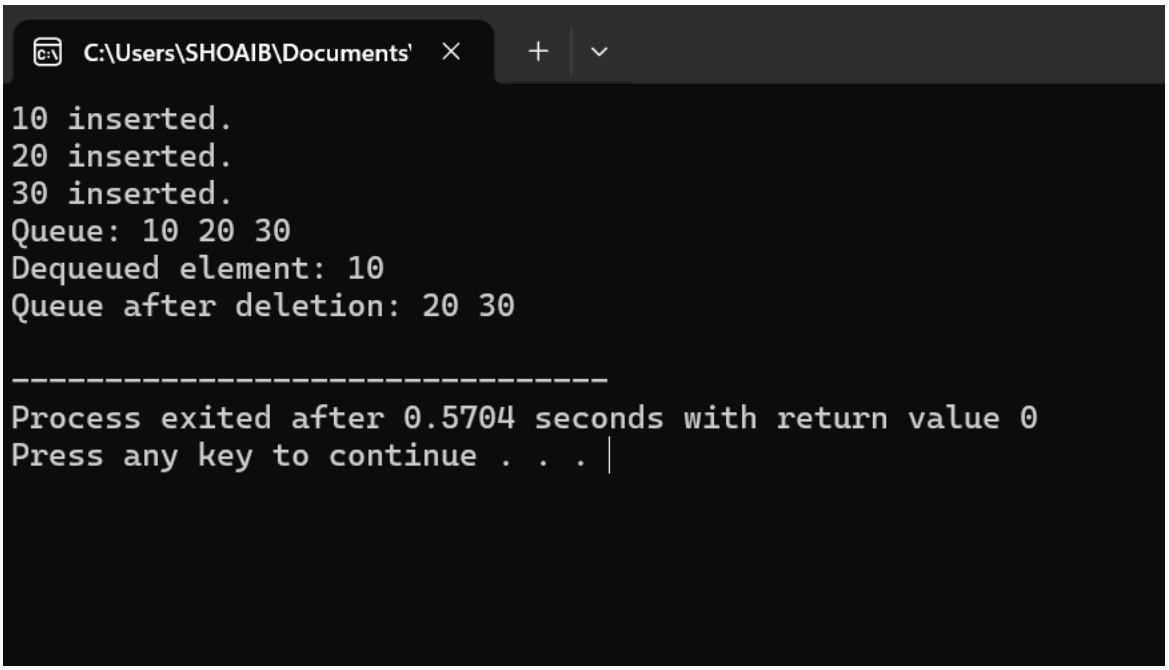
    cout << "Queue: ";
    q.display();

    cout << "Dequeued element: " << q.dequeue() << endl;

    cout << "Queue after deletion: ";
    q.display();
```

```
    return 0;
}
```

Output:



TASK#02:

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node* next;
    Node* prev;
```

```
};

class Deque
{
   Node* front;
   Node* rear;

public:
   Deque()
   {
      front = rear = NULL;
   }

   bool isEmpty()
   {
      return front == NULL;
   }

   void insert_at_beg(int value)
   {
      Node* p = new Node;
      p->data = value;
      p->prev = NULL;
```

```
    p->next = front;

    if (isEmpty())
        front = rear = p;
    else
    {
        front->prev = p;
        front = p;
    }

    cout << value << " inserted at front.\n";
}

void insert_at_end(int value)
{
    Node* p = new Node;
    p->data = value;
    p->next = NULL;
    p->prev = rear;

    if (isEmpty())
        front = rear = p;
    else
```

```
    {
        rear->next = p;
        rear = p;
    }

    cout << value << " inserted at rear.\n";
}

void delete_fr_front()
{
    if (isEmpty())
    {
        cout << "Deque is empty!\n";
        return;
    }

    Node* p = front;
    cout << "Deleted from front: " << p->data << endl;

    front = front->next;

    if (front == NULL)
        rear = NULL;
```

```
    else
        front->prev = NULL;

    delete p;
}

void delete_fr_rear()
{
    if (isEmpty())
    {
        cout << "Deque is empty!\n";
        return;
    }

    Node* p = rear;
    cout << "Deleted from rear: " << p->data << endl;

    rear = rear->prev;

    if (rear == NULL)
        front = NULL;
    else
        rear->next = NULL;
```

```
    delete p;
}

void show()
{
    if (isEmpty())
    {
        cout << "Deque is empty!\n";
        return;
    }

    Node* p = front;

    while (p != NULL)
    {
        cout << p->data << " ";
        p = p->next;
    }

    cout << endl;
}
```

```
    ~Deque()
    {
        while (front != NULL)
        {
            Node* p = front;
            front = front->next;
            delete p;
        }

        rear = NULL;
    }
};

int main()
{
    Deque d;

    d.insert_at_end(10);
    d.insert_at_end(20);
    d.insert_at_beg(5);

    cout << "Deque elements: ";
    d.show();
```

```
    d.delete_fr_front();
    d.delete_fr_rear();

    cout << "Remaining elements: ";
    d.show();

    return 0;
}
```

Output:

